You also want to avoid stress on the external service when it comes back online. When many actors restart, too many connections can be requested almost simultaneously—even with a backoff strategy—resulting in a load spike for the resource. To avoid this, a randomizer parameter is added to this configuration. The signature is as follows, and an example is shown at the end of the next section:

```
def restartWithBackoff(minBackoff: FiniteDuration, maxBackoff: FiniteDuration,
      randomFactor: Double): BackoffSupervisorStrategy
```

You can mix these strategies to create a custom strategy: a combination of multiple supervisions depending on the `catch` exceptions.

5.3.4 Custom strategy

You've seen the basics, but usually you don't want to handle all exceptions with the same strategy. The following snippet shows a more realistic scenario that combines different strategies depending on the exception. These exceptions are handled by different supervision strategies. To combine these strategies, you use the composition principle you've already seen, with one behavior inside the other:

```
Behavior.supervise(
  Behavior.supervise(
     Behaviors.supervise(
       behavior                ❶
     ).onFailure[ExceptionX](SupervisorStrategy.resume)
  ).onFailure[ExceptionY](SupervisorStrategy.restart)
).onFailure[ExceptionZ](SupervisorStrategy.stop)
```

❶ Behavior[T] goes here.